
Architecture Brief

ConnectRole

Current system architecture, data flow, and a practical roadmap toward a production-ready cloud platform

Prepared for	Felipe Marin
Project	AI mock interview coach built with Next.js, TypeScript, LLM APIs, and browser voice tooling
Purpose	Summarize the current architecture and outline the next infrastructure investments that would strengthen both the product and the resume story
Recommended direction	PostgreSQL + Prisma + AWS-first deployment, with Azure as a strong secondary path

This document is written as a planning memo rather than a formal specification. The goal is to explain how ConnectRole works today, where the architecture is still prototype-oriented, and which backend and cloud additions would make the project feel much closer to something a company would run in production.

1. What ConnectRole is today

ConnectRole is currently a polished product prototype with a light backend. The user experience is strong: the app accepts a resume, parses a target job posting, generates interview questions, scores spoken answers, and produces a final report. Architecturally, though, the application is still closer to an advanced single-session workflow than a full production system.

The frontend is built in Next.js with the App Router, and most of the experience lives in three client-driven screens: setup, interview, and results. On the server side, API routes provide narrow services such as parsing setup inputs, generating opening questions, evaluating interview turns, summarizing the session, and calling Google Cloud Text-to-Speech. The intelligence layer lives in reusable modules under the lib directory, which keeps the project organized and makes the business logic easier to evolve.

Current functional layers

- Presentation layer: React components guide the user through resume upload, interview practice, and results review.
- Application layer: Next.js API routes receive structured requests from the browser and return parsed or generated data.
- Intelligence layer: helper modules handle resume parsing, job parsing, question generation, turn evaluation, and report creation.
- External services: the project already communicates with LLM providers and Google Cloud Text-to-Speech.

Why this is a good starting point

From a hiring perspective, the existing app already shows real product thinking. It is not just a static portfolio site. It has a multi-step flow, API usage, typed data models, state transitions, and user-facing feedback logic. That gives it a strong base. The next leap is to turn the current transient workflow into a persistent, cloud-backed platform.

Core files that define the system

Setup flow	components/setup-screen.tsx and app/api/setup/parse/route.ts
Interview flow	components/interview-screen.tsx and app/api/interview/turn/route.ts
Results flow	components/results-screen.tsx and app/api/interview/summary/route.ts
Session storage	lib/session.ts
AI orchestration	lib/setup-intelligence.ts, lib/interview-brain.ts, and lib/interview-engine.ts

2. How data moves through the system

The current data flow is straightforward and easy to reason about. During setup, the browser extracts text from a PDF resume, gathers job-posting text from the user, and sends both payloads to a server route. The server parses and enriches those inputs, then returns structured resume and job objects. At that point, the browser becomes the main holder of state.

The interview page loads that saved setup data, asks questions, captures answers, and sends each turn to the evaluation route. The server evaluates the latest answer using the full client-supplied context, then returns the next question and coaching feedback. Finally, the results page assembles a final report, asks the server for a summary, and stores the completed report in the browser as well.

Current state ownership

Stage	Where data is created	Where data lives after creation
Setup	Browser + /api/setup/parse	sessionStorage in the browser
Interview turns	Browser + /api/interview/turn	sessionStorage in the browser
Results	Browser + /api/interview/summary	sessionStorage in the browser
Voice output	/api/tts	Returned directly as audio bytes

What this means in practice

- The app behaves well for one session on one device.
- There is no durable interview history if the tab is lost or the user changes devices.
- There is no server-side source of truth, so analytics and user accounts would be hard to add cleanly later.
- The client sends large context payloads repeatedly, which is acceptable for a prototype but not ideal for scale.

Most important architectural gap

The largest gap is not the lack of a cloud provider by itself. It is the lack of persistence and ownership boundaries. Right now, the browser effectively owns interview state. In a production system, the server should own interview sessions and the database should preserve them. Once that shift happens, cloud infrastructure starts to make much more sense because there is a real backend to host, monitor, protect, and scale.

3. Recommended target architecture

The best near-term architecture for ConnectRole is a server-centered design with PostgreSQL as the durable store, object storage for uploaded files, and managed cloud services for deployment and observability. This does not require a giant rewrite. It is more of a shift in responsibility: moving long-lived state out of sessionStorage and into backend services.

Recommended stack

Web application	Next.js App Router, keeping the existing frontend and API route structure
Database	PostgreSQL with Prisma for schema management and typed queries
File storage	Amazon S3 for resume uploads and future exported reports
Authentication	Auth.js or Amazon Cognito, depending on how cloud-native you want the story to be
Deployment	AWS Amplify or a container-based deployment path
Monitoring	CloudWatch, request logging, and basic error tracking

How the improved data model should work

1. A user signs in and starts a new interview preparation session.
2. The resume PDF is uploaded to object storage and its extracted text is saved in PostgreSQL.
3. The parsed job posting is stored as a first-class database record rather than only a temporary payload.
4. Starting an interview creates an interview_session record on the server.
5. Each answer inserts a new interview_turn row and updates the current server-side session state.

6. Completing the interview creates a final_report row that can be revisited later.

Why PostgreSQL is the right database here

This project is fundamentally relational. Users own resumes. Resumes and job postings connect to interview sessions. Sessions contain many turns. Sessions also produce one final report. That shape fits PostgreSQL naturally. It is also one of the most common databases employers expect students and new grads to understand, especially when paired with Prisma, SQL, and cloud deployment.

AWS versus Azure

Concern	AWS-first path	Azure-first path
Hosting	Amplify or container deployment	App Service or Static Web Apps
Database	RDS for PostgreSQL	Azure Database for PostgreSQL
Object storage	S3	Blob Storage
Identity	Cognito	Microsoft Entra
Monitoring	CloudWatch	Application Insights / Azure Monitor

If only one path is chosen, AWS is the stronger general-purpose choice for this project because it aligns well with full-stack deployment, object storage, and the broader job market. Azure remains a good secondary path, especially for companies built around Microsoft tooling.

4. Improvement roadmap for the next few months

A good improvement plan should create visible momentum quickly. The goal is not to boil the ocean. It is to turn the project into something that demonstrates backend ownership, database design, deployment maturity, and real cloud familiarity in a sequence that still feels achievable for a senior student.

Phase-by-phase plan

Phase	Main deliverable	Why it matters
1	Prisma + PostgreSQL integration	Introduces durable backend state and shows database modeling skill
2	User authentication	Turns anonymous sessions into user-owned history
3	Resume file storage in S3	Adds real cloud storage and separation of file data from relational data
4	Docker + local compose setup	Shows environment reproducibility and deployment readiness
5	AWS deployment + monitoring	Makes the project production-like and resume-ready

The highest-value implementation order

- Move setup, interview session, and final report persistence from sessionStorage to PostgreSQL.
- Introduce an interview session identifier so the browser can reload state from the server.
- Store the original PDF file separately from the parsed text and metadata.
- Containerize the application so local development and deployment use the same runtime assumptions.
- Add a basic CI pipeline that runs linting, type checks, and build validation.

How this helps your resume

Employers care less about whether a student merely used buzzwords and more about whether the system design choices make sense. If ConnectRole evolves into a project with PostgreSQL, Docker, object storage, authentication, and a managed AWS deployment, you can talk concretely about tradeoffs: why some data belongs in relational tables, why files belong in object storage, why Docker helps standardize environments, and how cloud services reduce operational burden.

That is exactly the kind of conversation that tends to differentiate a candidate who has built projects from a candidate who has only taken classes. The strongest version of this story is not just 'I used AWS.' It is 'I redesigned a client-held prototype into a persistent cloud-backed system and can explain each infrastructure choice.'

Final recommendation

The clearest next move is to make ConnectRole a database-backed AWS project. Start with PostgreSQL and Prisma, then add auth, S3, Docker, and deployment. That sequence improves the product, makes the architecture more correct, and gives you several highly marketable skills at once without losing the momentum of the app you already built.